Agentic AI Interview Questions

Section 1: Agentic AI Fundamentals

1. What is Agentic AI, and how does it differ from traditional AI systems?
2. What is an AI agent?
3. What are the main components of an AI agent?
4. What is the difference between a chatbot, an LLM application and an AI agent?
5. What is the difference between Generative AI and Agentic AI?
6. What are the characteristics of an autonomous AI agent?
7. What are the main applications of Agentic AI in real-world businesses?
8. What is the difference between a reactive agent and a goal-driven agent?
9. What are the advantages and limitations of Agentic AI?
10. What factors determine whether a problem requires an AI agent or a simpler LLM workflow?

Section 2: AI Agent Architecture

11. What is the typical architecture of an AI agent?
12. What roles do the LLM, tools, memory and orchestration layer play in an agent?
13. What is the perception–reasoning–action cycle?
14. What is the difference between an agent's internal state and its external environment?
15. What is the Observe–Think–Act cycle, and how is it implemented in an agent?
16. What is the difference between an agent loop and a fixed sequential workflow?
17. What is a goal-oriented agent, and how does it determine the next action?
18. What is the role of a controller or orchestrator in an agent system?
19. What is the difference between a single-agent architecture and a multi-agent architecture?
20. How would you design an agent that receives a user request, plans actions, executes tools and returns a validated result?

Section 3: Planning and Reasoning

21. What is task planning in Agentic AI?
22. What is task decomposition, and why is it important for complex tasks?
23. What is the difference between high-level planning and low-level execution?
24. What is ReAct (Reasoning and Acting), and how does it combine reasoning with tool use?
25. What is plan-and-execute architecture?
26. What is the difference between predefined planning and dynamic planning?
27. How can an agent revise its plan when a tool fails?
28. What is reflection in an AI agent, and how can it improve task completion?
29. What are the limitations of LLM-based planning and self-reflection?
30. How would you design an agent that breaks a research task into subtasks, verifies intermediate results and produces a final report?

Section 4: Tool Calling and Function Execution

31. What is tool calling in an AI agent?
32. How does an LLM decide which tool to call?
33. What is the difference between tool calling and a normal LLM response?
34. How do function schemas help an LLM call tools correctly?
35. What is the difference between a tool definition and a tool execution result?
36. How would you implement a Python tool that retrieves information from an external API?
37. How should an agent handle tool errors, timeouts and invalid arguments?
38. What is the difference between deterministic tools and LLM-based tools?
39. How would you prevent an agent from calling the same tool repeatedly without making progress?
40. How would you design and test an agent that can search the web, query a database and send an email?

Section 5: Memory and Context Management

41. What is memory in an AI agent, and why is it useful?
42. What is the difference between short-term memory and long-term memory?
43. What is the difference between conversation history and agent state?
44. What is working memory in an agent workflow?
45. How can vector databases support long-term semantic memory?
46. What is the difference between storing raw conversation history and storing summarised memories?
47. How can an agent retrieve relevant information from previous interactions?
48. What is context-window management, and why is it important for long-running agents?
49. How would you prevent stale, incorrect or contradictory memories from affecting an agent's decisions?
50. How would you build an agent that remembers user preferences across sessions while respecting privacy and data-retention requirements?

Section 6: Multi-Agent Systems

51. What is a multi-agent system, and when is it useful?
52. What is multi-agent collaboration?
53. What is task delegation in a multi-agent system?
54. What are the manager–worker, pipeline and peer-to-peer agent architectures?
55. What is the difference between centralised and decentralised agent orchestration?
56. How can specialised agents collaborate on a complex task?
57. What is the difference between sequential and parallel agent execution?
58. How do agents communicate their results and intermediate findings?
59. What problems can arise when multiple agents disagree or produce conflicting outputs?
60. How would you design a research system with separate researcher, analyst, fact-checker and report-writer agents?

Section 7: Agent Frameworks — LangChain and LangGraph

61. What is LangChain, and how does it support AI agent development?
62. What is the difference between a LangChain chain and an agent?
63. What is LangGraph, and why is graph-based orchestration useful?
64. What are nodes, edges and conditional edges in LangGraph?
65. What is shared state in LangGraph, and how is it updated?
66. How does a conditional edge help an agent decide whether to call a tool or return a final answer?
67. What are checkpoints in LangGraph, and why are they useful?
68. What is human-in-the-loop execution in LangGraph?
69. How can you implement retry logic, recursion limits and error handling in an agent graph?
70. When would you choose LangGraph over a simple LangChain agent loop?

Section 8: CrewAI and AutoGen

71. What is CrewAI, and what are its main components?
72. What is the difference between an agent, a task and a crew in CrewAI?
73. What are role, goal and backstory in a CrewAI agent definition?
74. What is the difference between sequential and hierarchical processes in CrewAI?
75. What are delegation and task dependencies in CrewAI?
76. What is Microsoft AutoGen, and what types of multi-agent workflows can it support?
77. How can agents exchange messages and coordinate tasks in AutoGen?
78. What is the difference between CrewAI and AutoGen in their approach to orchestration?
79. How would you choose between LangGraph, CrewAI and AutoGen for a production project?
80. How would you debug a multi-agent workflow in which agents repeatedly communicate but never produce a final result?

Section 9: Advanced Agent Design Patterns

81. What is the difference between a workflow, an agent and a hybrid workflow-agent system?
82. What is a supervisor-agent architecture?
83. What is a router agent, and how does it select the appropriate specialist?
84. What is the difference between handoff-based and supervisor-based agent coordination?
85. What is hierarchical task decomposition in multi-agent systems?
86. What is agentic RAG, and how does it differ from a conventional RAG pipeline?
87. What is the difference between corrective RAG and adaptive retrieval strategies?
88. What is an evaluator–optimiser pattern in agentic workflows?
89. What are parallelisation and fan-out/fan-in patterns in agent orchestration?
90. How would you combine deterministic workflows with autonomous decision-making to build a reliable customer-support agent?

Section 10: Agent Protocols, APIs and Integration

91. What is the Model Context Protocol (MCP), and how can it connect AI applications to external tools and data?
92. What are MCP clients, servers and tools?
93. What is the difference between MCP and ordinary function calling?
94. What is the difference between an MCP tool and an MCP resource?
95. What is an API, and how can an agent use a REST API as a tool?
96. What is the difference between synchronous and asynchronous tool execution?
97. How can an agent authenticate securely with external APIs?
98. What is structured output, and why is schema validation important in agent workflows?
99. How would you integrate an AI agent with a SQL database while preventing unsafe queries?
100. How would you design an agent that uses MCP-compatible tools alongside ordinary Python functions?

Section 11: Agent Evaluation and Reliability

101. What is agent evaluation, and how does it differ from evaluating a standalone LLM?
102. What is task-completion rate, and how can it be measured?
103. What is the difference between tool-selection accuracy and task-completion accuracy?
104. How would you evaluate the quality of an agent's plan?
105. What is trajectory evaluation in an agent system?
106. How can you detect unnecessary tool calls and inefficient agent loops?
107. How would you test an agent's behaviour when tools return incomplete or contradictory information?
108. What are deterministic tests, simulation tests and end-to-end tests for agents?
109. How can you evaluate a multi-agent system without relying entirely on an LLM-as-a-judge?
110. Which metrics would you monitor to compare two agent architectures for the same business task?

Section 12: Security, Guardrails and Responsible AI

111. What are the major security risks associated with autonomous AI agents?
112. What is prompt injection, and how can it manipulate an agent's behaviour?
113. How can indirect prompt injection occur through retrieved documents or web pages?
114. What is the principle of least privilege, and why is it important for agent tools?
115. How would you prevent an agent from executing destructive database operations without authorisation?
116. What is human approval, and when should an agent require it before taking an action?
117. How should an agent handle sensitive information, API keys and credentials?
118. How can sandboxing reduce the risks of executing agent-generated code?
119. What are the risks of giving an agent unrestricted access to email, files, databases or financial transactions?

120. How would you design audit logs and rollback mechanisms for an agent that performs actions in external systems?

Section 13: Production Deployment and Optimisation

121. What are the main challenges in deploying an AI agent to production?
122. How do you manage latency when an agent needs multiple LLM and tool calls?
123. What is the difference between concurrency and parallelism in agent execution?
124. How can caching reduce cost and latency in agent workflows?
125. How would you manage API rate limits across multiple agents?
126. How can you make tool execution idempotent to prevent duplicate side effects?
127. What is durable execution, and why is it important for long-running agents?
128. How would you handle interruptions, retries and recovery in a multi-step agent workflow?
129. How would you monitor token usage, tool failures, task completion, latency and cost in production?
130. How would you deploy a Python-based agent using FastAPI, Docker and a cloud platform?

Section 14: Advanced Agentic AI Concepts

131. What is autonomous goal pursuit, and how does it differ from executing a fixed plan?
132. What is multi-agent negotiation, and when might agents need to negotiate?
133. What is consensus in a multi-agent system, and what are its limitations?
134. What is a blackboard architecture in multi-agent systems?
135. What is a shared workspace, and how can it support collaboration between specialised agents?
136. What is the difference between model-based planning and LLM-generated planning?
137. How can reinforcement learning be used to improve agent policies?

138. What is the difference between an AI agent and a conventional software automation system?
139. What are the main limitations of current agentic systems in terms of reliability, observability and autonomy?
140. How would you design a long-running agent that performs research, waits for external events, resumes execution and asks for human approval when necessary?